

Chapter 10

Data Cleaning Features

There are a number of techniques for validating and converting data to native Python objects for subsequent analysis. This chapter guides you through three of these techniques, each appropriate for different kinds of data. The chapter moves on to the idea of standardization to transform unusual or atypical values into a more useful form. The chapter concludes with the integration of acquisition and cleansing into a composite pipeline.

This chapter will expand on the project in [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#). The following additional skills will be emphasized:

- CLI application extension and refactoring to add features.
- Pythonic approaches to validation and conversion.
- Techniques for uncovering key relationships.
- Pipeline architectures. This can be seen as a first step toward a processing **DAG (Directed Acyclic Graph)** in which various stages are connected.

We'll start with a description of the first project to expand on the previous chapters on processing. This will include some new **Pydantic** features to work with more complex data source fields.

10.1 Project 3.2: Validate and convert source fields

In [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#) we relied on the foundational behavior of the **Pydantic** package to convert numeric fields from the source text to Python types like `int`, `float`, and `Decimal`. In this chapter, we'll use a dataset that includes date strings so we can explore some more complex conversion rules.

This will follow the design pattern from the earlier project. It will use a distinct data set, however, and some unique data model definitions.

10.1.1 Description

This project's intent is to perform data validation, cleaning, and standardization. This project will expand on the features of the `pydantic` package to do somewhat more sophisticated data validations and conversions.

This new data cleaning application can be designed around a data set like https://tidesandcurrents.noaa.gov/tide_predictions.html. The tide predictions around the US include dates, but the fields are decomposed, and our data cleaning application needs to combine them.

For a specific example, see

<https://tidesandcurrents.noaa.gov/noaatideannual.html?id=8725769>.

Note that the downloaded `.txt` file is a tab-delimited CSV file with a very complicated multi-line header. This will require some sophisticated acquisition processing similar to the examples shown in [Chapter 3, Project 1.1: Data Acquisition Base Application](#).

An alternative example is the CO2 PPM — Trends in Atmospheric Carbon Dioxide data set, available at <https://datahub.io/core/co2-ppm>. This has dates that are provided in two forms: as a `year-month-day` string and as a decimal number. We can better understand this data if we can reproduce the decimal number value.

The second example data set is <https://datahub.io/core/co2-ppm/r/0.html>. This is an HTML file, requiring some acquisition processing similar to the examples from [Chapter 4, Data Acquisition Features: Web APIs and Scraping](#).

The use case for this cleaning application is identical to the description shown in [Chapter 9, Project 3.1: Data Cleaning Base Application](#). The acquired data — pure text, extracted from the source files — will be cleaned to create **Pydantic** models with fields of useful Python internal types.

We'll take a quick look at the tide table data on the <https://tidesandcurrents.noaa.gov> website.

```
NOAA/NOS/CO-OPS
Disclaimer: These data are based upon the latest information available ...
Annual Tide Predictions
StationName: EL JOBEAN, MYAKKA RIVER
State: FL
Stationid: 8725769
ReferencedToStationName: St. Petersburg, Tampa Bay
ReferencedToStationId: 8726520
HeightOffsetLow: * 0.83
HeightOffsetHigh: * 0.83
TimeOffsetLow: 116
TimeOffsetHigh: 98
Prediction Type: Subordinate
From: 20230101 06:35 - 20231231 19:47
Units: Feet and Centimeters
Time Zone: LST_LDT
Datum: MLLW
Interval Type: High/Low
```

```
Date Day Time Pred(Ft) Pred(cm) High/Low
2023/01/01 Sun 06:35 AM -0.13 -4 L
2023/01/01 Sun 01:17 PM 0.87 27 H
etc.
```

The data to be acquired has two interesting structural problems:

1. There's a 19-line preamble containing some useful metadata. Lines 2 to 18 have a format of a label and a value, for example, `State: FL`.
2. The data is tab-delimited CSV data. There appear to be six column titles. However, looking at the tab characters, there are eight columns of header data followed by nine columns of data.

The acquired data should fit the dataclass definition shown in the following fragment of a class definition:

```
from dataclasses import dataclass

@dataclass
class TidePrediction:
    date: str
    day: str
    time: str
    pred_ft: str
    pred_cm: str
    high_low: str

    @classmethod
    def from_row(
        cls: type["TidePrediction"],
        row: list[str]
    ) -> "TidePrediction":
        ...
```

The example omits the details of the `from_row()` method. If a CSV reader is used, this method needs to pick out columns from the CSV-format file, skipping over the generally empty columns. If regular expressions are used to parse the source lines, this method will use the groups from the match object.

Since this looks like many previous projects, we'll look at the distinct technical approach next.

10.1.2 Approach

The core processing of the data cleaning application should be — except for a few module changes — very similar to the earlier examples. For reference, see [Chapter 9, Project 3.1: Data Cleaning Base Application](#), specifically [Approach](#). This suggests that the `clean` module should have minimal changes from the earlier version.

The principle differences should be two different implementations of the `acquire_model` and the `analysis_model`. For the tide data example, a

class is shown in the [Description](#) section that can be used for the acquire model.

It's important to maintain a clear distinction between the acquired data, which is often text, and the data that will be used for later analysis, which can be a mixture of more useful Python object types.

The two-step conversion from source data to the interim acquired data format, and from the acquired data format to the clean data format can — sometimes — be optimized to a single conversion.

An optimization to combine processing into a single step can also make debugging more difficult.

We'll show one approach to defining the enumerated set of values for the state of the tide. In the source data, codes of 'H' and 'L' are used. The following class will define this enumeration of values:

```
from enum import StrEnum

class HighLow(StrEnum):
    high = 'H'
    low = 'L'
```

We'll rely on the enumerated type and two other annotated types to define a complete record. We'll return to the annotated types after showing the record as a whole first. A complete tide prediction record looks as follows:

```
import datetime
from typing import Annotated, TypeAlias

from pydantic import BaseModel
from pydantic.functional_validators import AfterValidator, BeforeValidator

# See below for the type aliases.

class TidePrediction(BaseModel):
    date: TideCleanDateTime
    pred_ft: float
    pred_cm: float
    high_low: TideCleanHighLow

    @classmethod
    def from_acquire_dataclass(
        cls,
        acquired: acquire_model.TidePrediction
    ) -> "TidePrediction":
        source_timestamp = f"{acquired.date} {acquired.time}"
        return TidePrediction(
            date=source_timestamp,
            pred_ft=acquired.pred_ft,
            pred_cm=acquired.pred_cm,
```

```

        high_low=acquired.high_low
    )

```

This shows how the source columns' `date` and `time` are combined into a single text value prior to validation. This is done by the `from_acquire_dataclass()` method, so it happens before invoking the `TidePrediction` constructor.

The `TideCleanDateTime` and `TideCleanHighLow` type hints will leverage annotated types to define validation rules for each of these attributes. Here are the two definitions:

```

TideCleanDateTime: TypeAlias = Annotated[
    datetime.datetime, BeforeValidator(clean_date)]
TideCleanHighLow: TypeAlias = Annotated[
    HighLow, BeforeValidator(lambda text: text.upper())]

```

The `TideCleanDateTime` type uses the `clean_date()` function to clean up the `date` string prior to any attempt at conversion. Similarly, the `TideCleanHighLow` type uses a lambda to transform the value to upper case before validation against the `HighLow` enumerated type.

The `clean_date()` function works by applying the one (and only) expected date format to the string value. This is not designed to be flexible or permissive. It's designed to confirm the data is an exact match against expectations.

The function looks like this:

```

def clean_date(v: str | datetime.datetime) -> datetime.datetime:
    match v:
        case str():
            return datetime.datetime.strptime(v, "%Y/%m/%d %I:%M %p")
        case _:
            return v

```

If the data doesn't match the expected format, the `strptime()` function will raise a `ValueError` exception. This will be incorporated into a `pydantic.ValidationError` exception that enumerates all of the errors encountered. The `match` statement will pass non-string values through to the **pydantic** handler for validation; we don't need to handle any other types.

This model can also be used for analysis of clean data. (See [Chapter 13, Project 4.1: Visual Analysis Techniques](#).) In this case, the data will already be a valid `datetime.datetime` object, and no conversion will need to be performed. The use of a type hint of `str | datetime.datetime` emphasizes the two types of values this method will be applied to.

This two-part "combine and convert" operation is broken into two steps to fit into the **Pydantic** design pattern. The separation follows the princi-

ple of minimizing complex initialization processing and creating class definitions that are more declarative and less active.

It's often helpful to keep the conversion steps small and separate.

Premature optimization to create a single, composite function is often a nightmare when changes are required.

For display purposes, the date, day-of-week, and time-of-day can be extracted from a single `datetime` instance. There's no need to keep many date-related fields around as part of the `TidePrediction` object.

The tide prediction is provided in two separate units of measure. For the purposes of this example, we retained the two separate values.

Pragmatically, the height in feet is the height in cm multiplied by $\frac{1}{30.48}$.

For some applications, where the value for height in feet is rarely used, a property might make more sense than a computed value. For other applications, where the two heights are both used widely, having both values computed may improve performance.

10.1.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for the application modules in the `tests` folder.
- Application to clean some acquired data and apply simple conversions to a few fields. Later projects will add more complex validation rules.

Many of these deliverables are described in previous chapters.

Specifically, [***Chapter 9, Project 3.1: Data Cleaning Base Application***](#) covers the basics of the deliverables for this project.

Unit tests for validation functions

The unique validators used by a Pydantic class need test cases. For the example shown, the validator function is used to convert two strings into a date.

Boundary Value Analysis suggests there are three equivalence classes for date conversions:

- Syntactically invalid dates. The punctuation or the number of digits is wrong.
- Syntactically valid, but calendrical invalid dates. The 30th of February, for example, is invalid, even when formatted properly.
- Valid dates.

The above list of classes leads to a minimum of three test cases.

Some developers like to explore each of the fields within a date, providing 5 distinct values: the lower limit (usually 1), the upper limit (e.g., 12 or 31), just below the limit (e.g., 0), just above the upper limit (e.g., 13 or 32), and a value that's in the range and otherwise valid. These additional test cases, however, are really testing the `strptime()` method of the `date-time` class. These cases are duplicate tests of the `datetime` module. These cases are *not* needed, since the `datetime` module already has plenty of test cases for calendrically invalid date strings.

Don't test the behavior of modules outside the application. Those modules have their own test cases.

In the next section, we'll look at a project to validate nominal data. This can be more complicated than validating ordinal or cardinal data.

10.2 Project 3.3: Validate text fields (and numeric coded fields)

For nominal data, we'll use **pydantic**'s technique of applying a validator function to the value of a field. In cases where the field contains a code consisting only of digits, there can be some ambiguity as to whether or not the value is a cardinal number. Some software may treat any sequence of digits as a number, dropping leading zeroes. This can lead to a need to use a validator to recover a sensible value for fields that are strings of digits, but not cardinal values.

10.2.1 Description

This project's intent is to perform data validation, cleaning, and standardization. This project will expand on the features of the **Pydantic** package to do somewhat more sophisticated data validation and conversion.

We'll continue working with a data set like

https://tidesandcurrents.noaa.gov/tide_predictions.html. The tide predictions around the US include dates, but the date is decomposed into three fields, and our data cleaning application needs to combine them.

For a specific example, see

<https://tidesandcurrents.noaa.gov/noaatideannual.html?id=8725769>.

Note that the downloaded `.txt` file is really a tab-delimited CSV file with a complex header. This will require some sophisticated acquisition processing similar to the examples shown in [Chapter 3, Project 1.1: Data Acquisition Base Application](#).

For data with a relatively small domain of unique values, a Python `enum` class is a very handy way to define the allowed collection of values. Using an enumeration permits simple, strict validation by `pydantic`.

Some data — like account numbers, as one example — have a large domain of values that may be in a state of flux. Using an `enum` class would mean transforming the valid set of account numbers into an enumerated type before attempting to work with any data. This may not be particularly helpful, since there's rarely a compelling need to confirm that an account number is valid; this is often a stipulation that is made about the data.

For fields like account numbers, there can be a need to validate potential values without an enumeration of all allowed values. This means the application must rely on patterns of the text to determine if the value is valid, or if the value needs to be cleaned to make it valid. For example, there may be a required number of digits, or check digits embedded within the code. In the case of credit card numbers, the last digit of a credit card number is used as part of confirmation that the overall number is valid. For more information, see

<https://www.creditcardvalidator.org/articles/luhn-algorithm>.

After considering some of the additional validations that need to be performed, we'll take a look at a design approach for adding more complicated validations to the cleaning application.

10.2.2 Approach

For reference to the general approach to this application, see [Chapter 9, Project 3.1: Data Cleaning Base Application](#), specifically [Approach](#).

The model can be defined using the `pydantic` package. This package offers two paths to validating string values against a domain of valid values. These alternatives are:

- Define an enumeration with all valid values.
- Define a regular expression for the string field. This has the advantage of defining very large domains of valid values, including *potentially* infinite domains of values.

Enumeration is an elegant solution that defines the list of values as a class. As shown earlier, it might look like this:

```
import enum

class HighLow(StrEnum):
    high = 'H'
    low = 'L'
```

This will define a domain of two string values, "L" and "H". This map provides easier-to-understand names, `Low` and `High`. This class can be used by `pydantic` to validate a string value.

An example of a case when we need to apply a `BeforeValidator` annotated type might be some tide data with lower-case "h" and "l" instead of proper upper-case "H" or "L". This allows the validator to clean the data **prior** to the built-in data conversion.

We might use an annotated type. It looked like this in the preceding example:

```
TideCleanHighLow: TypeAlias = Annotated[
    HighLow, BeforeValidator(lambda text: text.upper())]
```

The annotated type hint describes the base type, `HighLow`, and a validation rule to be applied before the **pydantic** conversion. In this case, it's a lambda to convert the text to upper case. We've emphasized the validation of enumerated values using an explicit enumeration because it is an important technique for establishing the complete set of allowed codes for a given attribute. The enumerated type's class definition is often a handy place to record notes and other information about the coded values.

Now that we've looked at the various aspects of the approach, we can turn our attention to the deliverables for this project.

10.2.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for the application modules in the `tests` folder.
- Application to clean source data in a number of fields.

Many of these deliverables are described in previous chapters.

Specifically, [***Chapter 9, Project 3.1: Data Cleaning Base Application***](#) covers the basics of the deliverables for this project.

Unit tests for validation functions

The unique validators used by a pydantic class need test cases. For the example shown, the validator function is used to validate the state of the tide. This is a small domain of enumerated values. There are three core kinds of test cases:

- Valid codes like `'H'` or `'L'`.
- Codes that can be reliably cleaned. For example, lower-case codes `'h'` and `'l'` are unambiguous. A data inspection notebook may reveal non-code values like `'High'` or `'Low'`, also. These can be reliably cleaned.
- Invalid codes like `''`, or `'9'`.

The domain of values that can be cleaned properly is something that is subject to a great deal of change. It's common to find problems and use an inspection notebook to uncover a new encoding when upstream applications change. This will lead to additional test cases, and then additional validation processing to make the test cases pass.

In the next project, we'll look at the situation where data must be validated against an externally defined set of values.

10.3 Project 3.4: Validate references among separate data sources

In [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#) we relied on the foundational behavior of Pydantic to convert fields from source text to Python types. This next project will look at a more complicated validation rule.

10.3.1 Description

This project's intent is to perform data validation, cleaning, and standardization. This project will expand on the features of the `pydantic` package to do somewhat more sophisticated data validations and conversions.

Data sets in <https://data.census.gov> have **ZIP Code Tabulation Areas (ZCTAs)**. For certain regions, these US postal codes can (and should) have leading zeroes. In some variations on this data, however, the ZIP codes get treated as numbers and the leading zeroes get lost.

Data sets at <https://data.census.gov> have information about the city of Boston, Massachusetts, which has numerous US postal codes with leading zeroes. The Food Establishment Inspections available at <https://data.boston.gov/group/permitting> provides insight into Boston-area restaurants. In addition to postal codes (which are nominal data), this data involves numerous fields that contain nominal data as well as ordinal data.

For data with a relatively small domain of unique values, a Python `enum` class is a very handy way to define the allowed collection of values. Using an enumeration permits simple, strict validation by **Pydantic**.

Some data — like account numbers, as one example — have a large domain of values that may be in a state of flux. Using an `enum` class would mean transforming the valid set of account numbers into an enum before attempting to work with any data. This may not be particularly helpful, since there's rarely a compelling need to confirm that an account number is valid; this is often a simple stipulation that is made about the data.

This leads to a need to validate potential values without an enumeration of the allowed values. This means the application must rely on patterns of

the text to determine if the value is valid, or if the value needs to be cleaned to make it valid.

When an application cleans postal code data, there are two distinct parts to the cleaning:

1. Clean the postal code to have the proper format. For US ZIP codes, it's generally 5 digits. Some codes are 5 digits, a hyphen, and 4 more digits.
2. Compare the code with some master list to be sure it's a meaningful code that references an actual post office or location.

It's important to keep these separate since the first step is covered by the previous project, and doesn't involve anything terribly complicated. The second step involves some additional processing to compare a given record against a master list of allowed values.

10.3.2 Approach

For reference to the general approach to this application, see [Chapter 9, Project 3.1: Data Cleaning Base Application](#), specifically [Approach](#).

When we have nominal values that must refer to external data, we can call these “foreign keys.” They're references to an external collection of entities for which the values are primary keys. An example of this is a postal code. There is a defined list of valid postal codes; the code is a primary key in this collection. In our sample data, the postal code is a foreign key reference to the defining collection of postal codes.

Other examples include country codes, US state codes, and US phone system area codes. We can write a regular expression to describe the potential domain of key values. For US state codes, for example, we can use the regular expression `r'\w\w'` to describe state codes as having two letters. We could narrow this domain slightly using `r'[A-Z]{2}'` to require the state code use upper-case letters only. There are only 50 state codes, plus a few territories and districts; limiting this further would make for a very long regular expression.

The confounding factor here is when the primary keys need to be loaded from an external source — for example, a database. In this case, the simple `@validator` method has a dependency on external data. Further, this data must be loaded prior to any data cleaning activities.

We have two choices for gathering the set of valid key values:

- Create an `Enum` class with a list of valid values.
- Define a `@classmethod` to initialize the pydantic class with valid values.

For example, <https://data.opendatasoft.com> has a useful list of US zip codes. See the URL

<https://data.opendatasoft.com/api/explore/v2.1/catalog/datasets/georef-united-states-of-america-zc-point@public/exports/csv> for US Zip Codes Points, United States of America. This is a file that can be downloaded and transformed into an enum or used to initialize a class. The `Enum` class creation is a matter of creating a list of two tuples with the label and the value for the enumeration. The `Enum` definition can be built with code like the following example:

```
import csv
import enum
from pathlib import Path

def zip_code_values() -> list[tuple[str, str]]:
    source_path = Path.home() / "Downloads" / "georef-united-states-of-america-zc-point@public.csv"
    with source_path.open(encoding='utf_8_sig') as source:
        reader = csv.DictReader(source, delimiter=',')
        values = [
            (f"ZIP_{zip['Zip Code']:0>5s}", f"{zip['Zip Code']:0>5s}")
            for zip in reader
        ]
    return values

ZipCode = enum.Enum("ZipCode", zip_code_values())
```

This will create an enumerated class, `ZipCode`, from the approximately 33,000 ZIP codes in the downloaded source file. The enumerated labels will be Python attribute names similar to `ZIP_75846`. The values for these labels will be the US postal codes, for example, `'75846'`. The `":0>5s"` string format will force in leading zeroes where needed.

The `zip_code_values()` function saves us from writing 30,000 lines of code to define the enumeration class, `ZipCode`. Instead, this function reads 30,000 values, creating a list of pairs used to create an `Enum` subclass.

The odd encoding of `utf_8_sig` is necessary because the source file has a leading **byte-order mark (BOM)**. This is unusual but permitted by Unicode standards. Other data sources for ZIP codes may not include this odd artifact. The encoding gracefully ignores the BOM bytes.

The unusual encoding of `utf_8_sig` is a special case because this file happens to be in an odd format.

There are a large number of encodings for text. While UTF-8 is popular, it is not universal.

When unusual characters appear, it's important to find the source of the data and ask what encoding they used.

In general, it's impossible to uncover the encoding given a sample file.

There are a large number of valid byte code mappings that overlap between ASCII, CP1252, and UTF-8.

This design requires the associated data file. One potential improvement is to create a Python module from the source data.

Using the **Pydantic** functional validators uses a similar algorithm to the one shown above. The validation initialization is used to build an object that retains a set of valid values. We'll start with the goal of a small model using annotated types. The model looks like this:

```
import csv
from pathlib import Path
import re
from typing import TextIO, TypeAlias, Annotated

from pydantic import BaseModel, Field
from pydantic.functional_validators import BeforeValidator, AfterValidator

# See below for the type aliases.

ValidZip: TypeAlias = Annotated[
    str,
    BeforeValidator(zip_format_valid),
    AfterValidator(zip_lookup_valid)]

class SomethingWithZip(BaseModel):
    # Some other fields
    zip: ValidZip
```

The model relies on the `ValidZip` type. This type has two validation rules: before any conversion, a `zip_format_valid()` function is applied, and after conversion, a `zip_lookup_valid()` function is used.

We've only defined a single field, `zip`, in this **Pydantic** class. This will let us focus on the validation-by-lookup design. A more robust example, perhaps based on the Boston health inspections shown above, would have a number of additional fields reflecting the source data to be analyzed.

The before validator function, `zip_format_valid()`, compares the ZIP code to a regular expression to ensure that it is valid:

```
def zip_format_valid(zip: str) -> str:
    assert re.match(r'\d{5}|\d{5}-d{4}', zip) is not None,
        f"ZIP invalid format {zip!r}"
    return zip
```

The `zip_format_valid()` can be expanded to use an f-string like `f"{zip:0>5s}"` to reformat a ZIP code that's missing the leading zeroes. We'll leave this for you to integrate into this function.

The after validator function is a callable object. It's an instance of a class that defines the `__call__()` method.

Here's the core class definition, and the creation of the instance:

```
class ZipLookupValidator:
    """Compare a code against a list."""
    def __init__(self) -> None:
        self.zip_set: set[str] = set()

    def load(self, source: TextIO) -> None:
        reader = csv.DictReader(source, delimiter=',')
        self.zip_set = {
            f"{zip['Zip Code']:0>5s}"
            for zip in reader
        }

    def __call__(self, zip: str) -> str:
        if zip in self.zip_set:
            return zip
        raise ValueError(f"invalid ZIP code {zip}")

zip_lookup_valid = ZipLookupValidator()
```

This will define the `zip_lookup_valid` callable object. Initially, there's new value for the internal `self.zip_set` attribute. This must be built using a function that evaluates `zip_lookup_valid.load(source)`. This will populate the set of valid values.

We've called this function `prepare_validator()` and it looks like this:

```
def prepare_validator() -> None:
    source_path = (
        Path.home() / "Downloads" /
        "georef-united-states-of-america-zc-point@public.csv"
    )
    with source_path.open(encoding='utf_8_sig') as source:
        zip_lookup_valid.load(source)
```

This idea of a complex validation follows the SOLID design principle. It separates the essential work of the `SomethingWithZip` class from the `ValidZip` type definition.

Further, the `ValidZip` type depends on a separate class, `ZipLookupValidator`, which handles the complications of loading data. This separation makes it somewhat easier to change validation files, or change the format of the data used for validation without breaking the `SomethingWithZip` class and the applications that use it. Further, it provides a reusable type, `ValidZip`. This can be used for multiple fields of a model, or multiple models.

Having looked at the technical approach, we'll shift to looking at the deliverables for this project.

10.3.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for the application modules in the `tests` folder.
- Application to clean and validate data against external sources.

Many of these deliverables are described in previous chapters.

Specifically, [*Chapter 9, Project 3.1: Data Cleaning Base Application*](#) covers the basics of the deliverables for this project.

Unit tests for data gathering and validation

The unique validators used by a **Pydantic** class need test cases. For the example shown, the validator function is used to validate US ZIP codes. There are three core kinds of test cases:

- Valid ZIP codes with five digits that are found in the ZIP code database.
- Syntactically valid ZIP codes with five digits that are **not** found in the ZIP code database.
- Syntactically invalid ZIP that don't have five digits, or can't — with the addition of leading zeroes — be made into valid codes.

10.4 Project 3.5: Standardize data to common codes and ranges

Another aspect of cleaning data is the transformation of raw data values into standardized values. For example, codes in use have evolved over time, and older data codes should be standardized to match new data codes. The notion of standardizing values can be a sensitive topic if critical information is treated as an outlier and rejected or improperly standardized.

We can also consider imputing new values to fill in for missing values as a kind of standardization technique. This can be a necessary step when dealing with missing data or data that's likely to represent some measurement error, not the underlying phenomenon being analyzed.

This kind of transformation often requires careful, thoughtful justification. We'll show some programming examples. The deeper questions of handling missing data, imputing values, handling outliers, and other standardization operations are outside the scope of this book.

See <https://towardsdatascience.com/6-different-ways-to-compensate-for-missing-values-data-imputation-with-examples-6022d9ca0779> for an overview of some ways to deal with missing or invalid data.

10.4.1 Description

Creating standardized values is at the edge of data cleaning and validation. These values can be described as "derived" values, computed from existing values.

There are numerous kinds of standardizations; we'll look at two:

1. Compute a standardized value, or Z-score, for cardinal data. For a normal distribution, the Z-scores have a mean of 0, and a standard deviation of 1. It permits comparing values measured on different scales.
2. Collapse nominal values into a single standardized value. For example, replacing a number of historical product codes with a single, current product code.

The first of these, computing a Z-score, rarely raises questions about the statistical validity of the standardized value. The computation, $Z = \frac{x - \mu}{\sigma}$, is well understood and has known statistical properties.

The second standardization, replacing nominal values with a standardized code, can be troubling. This kind of substitution may simply correct errors in the historical record. It may also obscure an important relationship. It's not unusual for a data inspection notebook to reveal outliers or erroneous values in a data set that needs to be standardized.

Enterprise software may have unrepaired bugs. Some business records can have unusual code values that map to other code values.

Of course, the codes in use may shift over time.

Some records may have values that reflect two eras: pre-repair and post-repair. Worse, of course, there may have been several attempts at a repair, leading to more nuanced timelines.

For this project, we need some relatively simple data. The Anscombe's Quartet data will do nicely as examples from which derived Z-scores can be computed. For more information, see [*Chapter 3, Project 1.1: Data Acquisition Base Application*](#).

The objective is to compute standardized values for the two values that comprise the samples in the Anscombe's Quartet series. When the data has a normal distribution, these derived, standardized Z-scores will have a mean of zero and a standard deviation of one. When the data does not have a normal distribution, these values will diverge from the expected values.

10.4.2 Approach

For reference to the general approach to this application, see [Chapter 9](#), [Project 3.1: Data Cleaning Base Application](#), specifically [Approach](#).

To replace values with preferred standardized values, we've seen how to clean bad data in previous projects. See, for example, [Project 3.3: Validate text fields \(and numeric coded fields\)](#).

For Z-score standardization, we'll be computing a derived value. This requires knowing the mean, μ , and standard deviation, σ , for a variable from which the Z-score can be computed.

This computation of a derived value suggests there are the following two variations on the analytical data model class definitions:

- An “initial” version, which lacks the Z-score values. These objects are incomplete and require further computation.
- A “final” version, where the Z-score values have been computed. These objects are complete.

There are two common approaches to handling this distinction between incomplete and complete objects:

- The two classes are distinct. The complete version is a subclass of the incomplete version, with additional fields defined.
- The derived values are marked as optional. The incomplete version starts with `None` values.

The first design is a more conventional object-oriented approach. The formality of a distinct type to clearly mark the state of the data is a significant advantage. The extra class definition, however, can be seen as clutter, since the incomplete version is transient data that doesn't create enduring value. The incomplete records live long enough to compute the complete version, and the file can then be deleted.

The second design is sometimes used for functional programming. It saves the subclass definition, which can be seen as a slight simplification.

```
from pydantic import BaseModel

class InitialSample(BaseModel):
    x: float
    y: float

class SeriesSample(InitialSample):
    z_x: float
    z_y: float

    @classmethod
    def build_sample(cls, m_x: float, s_x: float,
                    m_y: float, s_y: float, init:
                    InitialSample) -> "SeriesSample":
```

```

return SeriesSample(
    x=init.x, y=init.y,
    z_x=(init.x - m_x) / s_x,
    z_y=(init.y - m_y) / s_y
)

```

These two class definitions show one way to formalize the distinction between the initially cleaned, validated, and converted data, and the complete sample with the standardized Z-scores present for both of the variables.

This can be handled as three separate operations:

1. Clean and convert the initial data, writing a temporary file of the `InitialSample` instances.
2. Read the temporary file, computing the means and standard deviations of the variables.
3. Read the temporary file again, building the final samples from the `InitialSample` instances and the computed intermediate values.

A sensible optimization is to combine the first two steps: clean and convert the data, accumulating values that can be used to compute the mean and standard deviation. This is helpful because the `statistics` module expects a sequence of objects that might not fit in memory. The mean, which involves a sum and a count, is relatively simple. The standard deviation requires accumulating a sum and a sum of squares.

$$m_x = \frac{\sum x}{n}$$

The mean of x , m_x , is the sum of the x values divided by the count of x values, shown as n .

$$s_x = \sqrt{\frac{\sum x^2 - \frac{(\sum x)^2}{n}}{n - 1}}$$

The standard deviation of x , s_x , uses the sum of x^2 , the sum of x , and the number of values, n .

This formula for the standard deviation has some numeric stability issues, and there are variations that are better designs. See

https://en.wikipedia.org/wiki/Algorithms_for_calculating_variance.

We'll define a class that accumulates the values for mean and variance. From this, we can compute the standard deviation.

```

import math

class Variance:
    def __init__(self):
        self.k: float | None = None
        self.e_x = 0.0
        self.e_x2 = 0.0

```

```

        self.n = 0

    def add(self, x: float) -> None:
        if self.k is None:
            self.k = x
        self.n += 1
        self.e_x += x - self.k
        self.e_x2 += (x - self.k) ** 2

    @property
    def mean(self) -> float:
        return self.k + self.e_x / self.n

    @property
    def variance(self) -> float:
        return (self.e_x2 - self.e_x ** 2 / self.n) / (self.n - 1)

    @property
    def stdev(self) -> float:
        return math.sqrt(self.variance)

```

This `variance` class performs an incremental computation of mean, standard deviation, and variance. Each individual value is presented by the `add()` method. After all of the data has been presented, the properties can be used to return the summary statistics.

It's used as shown in the following snippet:

```

var_compute = Variance()
for d in data:
    var_compute.add(d)

print(f"Mean = {var_compute.mean}")
print(f"Standard Deviation = {var_compute.stdev}")

```

This provides a way to compute the summary statistics without using a lot of memory. It permits the optimization of computing the statistics the first time the data is seen. And, it reflects a well-designed algorithm that is numerically stable.

Now that we've explored the technical approach, it's time to look at the deliverables that must be created for this project.

10.4.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for the application modules in the `tests` folder.
- Application to clean the acquired data and compute derived standardized Z-scores.

Many of these deliverables are described in previous chapters.

Specifically, [***Chapter 9, Project 3.1: Data Cleaning Base Application***](#) covers the basics of the deliverables for this project.

Unit tests for standardizing functions

There are two parts of the standardizing process that require unit tests. The first is the incremental computation of mean, variance, and standard deviation. This must be compared against results computed by the `statistics` module to assure that the results are correct. The `pytest.approx` object (or the `math.isclose()` function) are useful for asserting the incremental computation matches the expected values from the standard library module.

Additionally, of course, the construction of the final sample, including the standardized Z-scores, needs to be tested. The test case is generally quite simple: a single value with a given x, y, mean of x, mean of y, the standard deviation of x, and the standard deviation of y need to be converted from the incomplete form to the complete form. The computation of the derived values is simple enough that the expected results can be computed by hand to check the results.

It's important to test this class, even though it seems very simple. Experience suggests that these seemingly simple classes are places where a `+` replaces a `-` and the distinction isn't noticed by people inspecting the code. This kind of small mistake is best found with a unit test.

Acceptance test

The acceptance test suite for this standardization processing will involve a main program that creates two output files. This suggests the after-scenario cleanup needs to ensure the intermediate file is properly removed by the application.

The cleaning application could use the `tempfile` module to create a file that will be deleted when closed. This is quite reliable, but it can be difficult to debug very obscure problems if the file that reveals the problems is automatically deleted. This doesn't require any additional acceptance test. Then step to be sure the file is removed, since we don't need to test the `tempfile` module.

The cleaning application can also create a temporary file in the current working directory. This can be unlinked for normal operation, but left in place for debugging purposes. This will require at least two scenarios to be sure the file is removed normally, and be sure the file is retained to support debugging.

The final choice of implementation — and the related test scenarios — is left to you.

10.5 Project 3.6: Integration to create an acquisition pipeline

In [User experience](#), we looked at the two-step user experience. One command is used to acquire data. After this, a second command is used to clean the data. An alternative user experience is a single shell pipeline.

10.5.1 Description

The previous projects in this chapter have decomposed the cleaning operation into two distinct steps. There's another, very desirable user experience alternative.

Specifically, we'd like the following to work, also:

```
% python src/acquire.py -s Series_1Pair --csv source.csv | python src/clean.py -o analysis/Se
```

The idea is to have the **acquire** application write a sequence of NDJSON objects to standard output. The **clean** application will read the sequence of NDJSON objects from standard input. The two applications will run concurrently, passing data from process to process.

For very large data sets, this can reduce the processing time. Because of the overhead in serializing Python objects to JSON text and deserializing Python objects from the text, the pipeline will not run in half the time of the two steps executed serially.

Multiple extractions

In the case of CSV extraction of Anscombe Quartet data, we have an **acquire** application that's capable of creating four files concurrently. This doesn't fit well with the shell pipeline. We have two architectural choices for handling this.

One choice is to implement a "fan-out" operation: the **acquire** program fans out data to four separate clean applications. This is difficult to express as a collection of shell pipelines. To implement this, a parent application uses `concurrent.futures`, queues, and processing pools. Additionally, the **acquire** program would need to write to shared queue objects, and the **clean** program would read from a shared queue.

The alternative is to process only one of the Anscombe series at a time. Introducing a `-s Series_1Pair` argument lets the user name a class that can extract a single series from the source data. Processing a single series at a time permits a pipeline that can be readily described as a shell command.

This concept is often necessary to disentangle enterprise data. It's common for enterprise applications — which often evolve organically — to

have values from distinct problem domains as parts of a common record.

We'll turn to the technical approach in the next section.

10.5.2 Approach

For reference to the general approach to this application, see [Chapter 9, Project 3.1: Data Cleaning Base Application](#), specifically [Approach](#).

Writing the standard output (and reading from standard input) suggests that these applications will have two distinct operating modes:

- Opening a named file for output or input.
- Using an existing, open, unnamed file — often a pipe created by the shell — for output or input.

This suggests that the bulk of the design for an application needs to focus on open file-like objects. These are often described by the type hint of `TextIO`: they are files that can read (or write) text.

The top-level `main()` function must be designed either to open a named file, or to provide `sys.stdout` or `sys.stdin` as argument values. The various combinations of files are provided to a function that will do the more useful work.

This pattern looks like the following snippet:

```
if options.output:
    with options.output.open('w') as output:
        process(options.source, output)
else:
    process(options.source, sys.stdout)
```

The `process()` function is either given a file opened by a context manager, or the function is given the already open `sys.stdout`.

The ability for a Python application to be part of a shell pipeline is a significant help in creating larger, more sophisticated composite processes. This higher-level design effort is sometimes called “Programming In The Large.”

Being able to read and write from pipelines was a core design feature of the Unix operating system and continues to be central to all of the various GNU/Linux variants.

This pipeline-aware design has the advantage of being slightly easier to unit test. The `process()` function's output argument value can be an `io.StringIO` object. When using a `StringIO` object, the file processing is simulated entirely in memory, leading to faster, and possibly simpler, tests.

This project sets the stage for a future project. See [Chapter 12, Project 3.8: Integrated Data Acquisition Web Service](#) for a web service that can leverage this pipeline.

Consider packages to help create a pipeline

A Python application to create a shell pipeline can involve a fair amount of programming to create two subprocesses that share a common buffer. This is handled elegantly by the shell.

An alternative is <https://cgarciae.github.io/pypeln/>. The **PypeLn** package is an example of a package that wraps the `subprocess` module to make it easier for a parent application to create a pipeline that executes the two child applications: **acquire** and **clean**.

Using a higher-level Python application to start the acquire-to-clean pipeline avoids the potential pitfalls of shell programming. It permits Python programs with excellent logging and debugging capabilities.

Now that we've seen the technical approach, it's appropriate to review the deliverables.

10.5.3 Deliverables

This project has the following deliverables:

- Documentation in the `docs` folder.
- Acceptance tests in the `tests/features` and `tests/steps` folders.
- Unit tests for the application modules in the `tests` folder.
- Revised applications that can be processed as a pipeline of two concurrent processes.

Many of these deliverables are described in previous chapters. Specifically, [Chapter 9, Project 3.1: Data Cleaning Base Application](#) covers the basics of the deliverables for this project.

Acceptance test

The acceptance test suite needs to confirm the two applications can be used as stand-alone commands, as well as used in a pipeline. One technique for confirming the pipeline behavior is to use shell programs like `cat` to provide input that mocks the input from another application.

For example, the `When` step may execute the following kind of command:

```
cat some_mock_file.ndj | python src/clean.py -o analysis/some_file.ndj
```

The **clean** application is executed in a context where it is part of an overall pipeline. The head of the pipeline is not the **acquire** application; we've used the `cat some_mock_file.ndj` command as a useful mock for the

other application's output. This technique permits a lot of flexibility to test applications in a variety of shell contexts.

Using a pipeline can permit some helpful debugging because it disentangles two complicated programs into two smaller programs. The programs can be built, tested, and debugged in isolation.

10.6 Summary

This chapter expanded in several ways on the project in [Chapter 9, Project 3.1: Data Cleaning Base Application](#). The following additional processing features were added:

- Pythonic approaches to validation and conversion of cardinal values.
- Approaches to validation and conversion of nominal and ordinal values.
- Techniques for uncovering key relationships and validating data that must properly reference a foreign key.
- Pipeline architectures using the shell pipeline.

10.7 Extras

Here are some ideas for you to add to these projects.

10.7.1 Hypothesis testing

The computations for mean, variance, standard deviation, and standardized Z-scores involve floating-point values. In some cases, the ordinary truncation errors of float values can introduce significant numeric instability. For the most part, the choice of a proper algorithm can ensure results are useful.

In addition to basic algorithm design, additional testing is sometimes helpful. For numeric algorithms, the **Hypothesis** package is particularly helpful. See <https://hypothesis.readthedocs.io/en/latest/>.

Looking specifically at [Project 3.5: Standardize data to common codes and ranges](#), the [Approach](#) section suggests a way to compute the variance. This class definition is an excellent example of a design that can be tested effectively by the Hypothesis module to confirm that the results of providing a sequence of three known values produces the expected results for the count, sum, mean, variance, and standard deviation.

10.7.2 Rejecting bad data via filtering (instead of logging)

In the examples throughout this chapter, there's been no in-depth mention of what to do with data that raises an exception because it cannot be processed. There are three common choices:

1. Allow the exception to stop processing.
2. Log each problem row as it is encountered, discarding it from the output.
3. Write the faulty data to a separate output file so it can be examined with a data inspection notebook.

The first option is rather drastic. This is useful in some data cleaning applications where there's a reasonable expectation of very clean, properly curated data. In some enterprise applications, this is a sensible assumption, and invalid data is the cause for crashing the application and sorting out the problems.

The second option has the advantage of simplicity. A `try: / except:` block can be used to write log entries for faulty data. If the volume of problems is small, then locating the problems in the log and resolving them may be appropriate.

The third option is often used when there is a large volume of questionable or bad data. The rows are written to a file for further study.

You are encouraged to implement this third strategy: create a separate output file for rejected samples. This means creating acceptance tests for files that will lead to the rejection of at least one faulty row.

10.7.3 Disjoint subentities

An even more complicated data validation problem occurs when the source documents don't reflect a single resulting dataclass. This often happens when disjoint subtypes are merged into a single data set. This kind of data is a union of the disjoint types. The data must involve a "discriminator" field that shows which type of object is being described.

For example, we may have a few fields with date, time, and document ID that are common to all samples. In addition to those fields, a `document_type` field provides a set of codes to discriminate between the different kinds of invoices and different kinds of payments.

In this case, a conversion function involves two stages of conversions:

- Identify the subtype. This may involve converting the common fields and the discriminator field. The work will be delegated to a subtype-specific conversion for the rest of the work.
- Convert each subtype. This may involve a family of functions associated with each of the discriminator values.

This leads to a function design as shown in the activity diagram in [*Figure 10.1*](#).

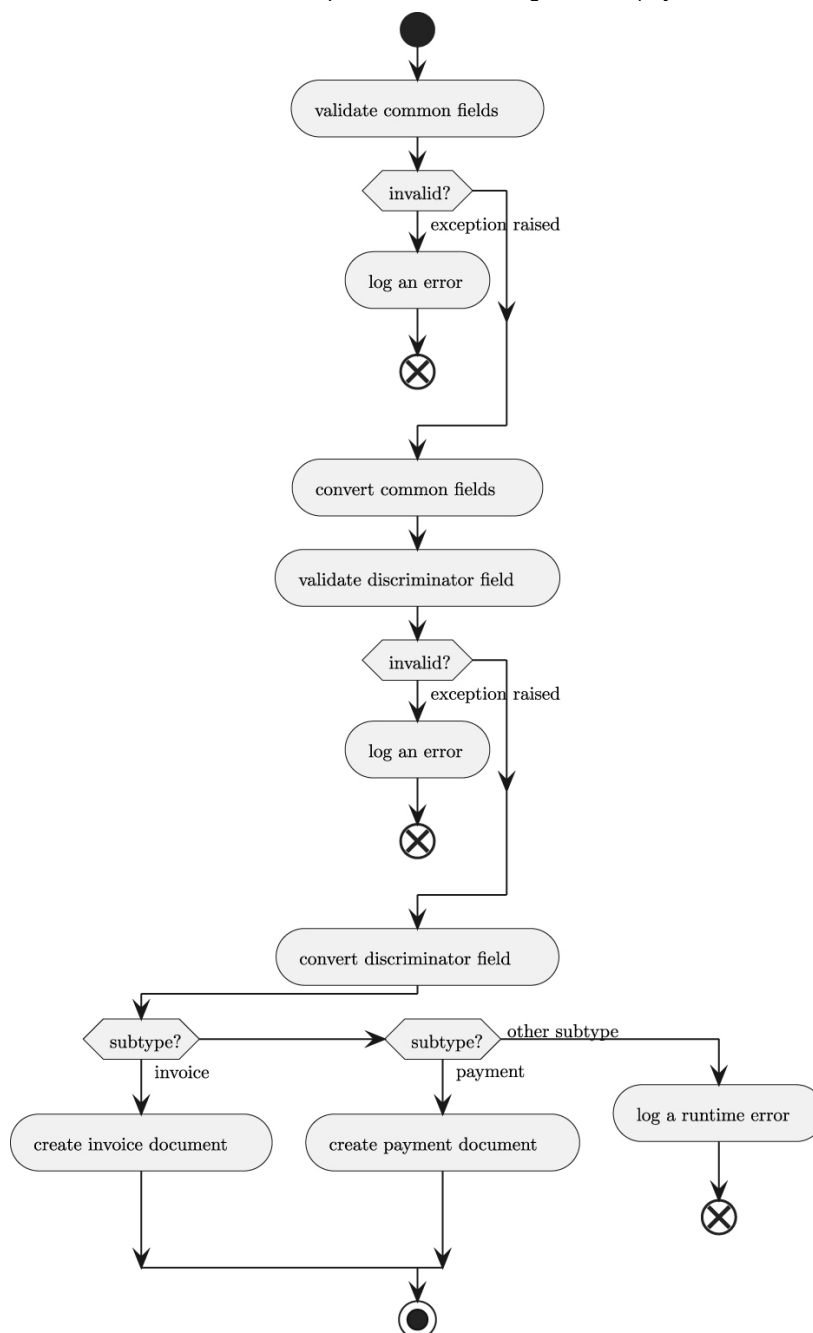


Figure 10.1: Subentity validation

10.7.4 Create a fan-out cleaning pipeline

There are two common alternatives for concurrent processing of a **acquire** and **clean** application:

- A shell pipeline that connects the **acquire** application to the **clean** application. These two subprocesses run concurrently. Each ND JSON line written by the **acquire** application is immediately available for processing by the **clean** application.
- A pool of workers, managed by `concurrent.futures`. Each ND JSON line created by the **acquire** application is placed in a queue for one of the workers to consume.

The shell pipeline is shown in [Figure 10.2](#).

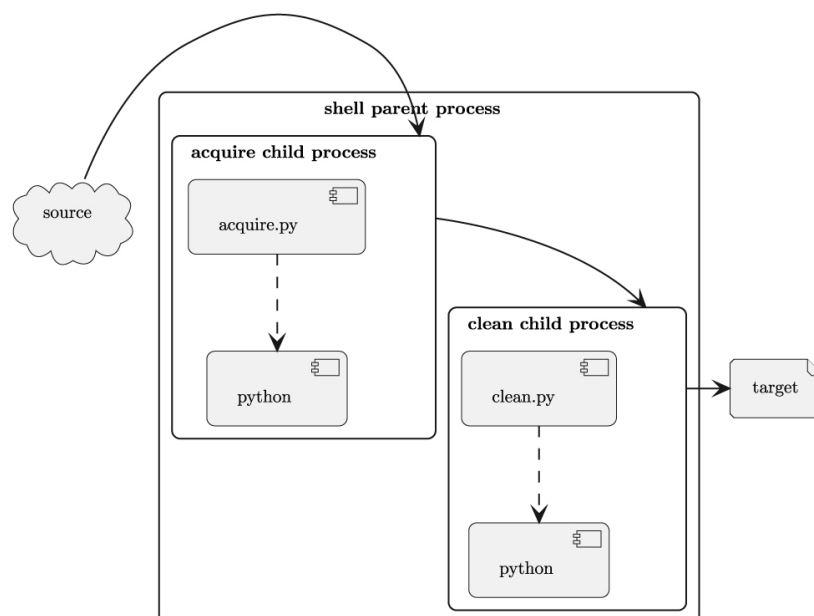


Figure 10.2: Components of a shell pipeline

The shell creates two child process with a shared buffer between them. For the acquire child process, the shared buffer is `sys.stdout`. For the clean child process, the shared buffer is `sys.stdin`. As the two applications run, each byte written is available to be read.

We've included explicit references to the Python runtime in these diagrams. This can help clarify how our application is part of the overall Python environment.

The pipeline creation is an elegant feature of the shell, and can be used to create complex sequences of concurrent processing. This is a handy way to think of decomposing a large collection of transformations into a number of concurrent transformations.

In some cases, the pipeline model isn't ideal. This is often the case when we need asymmetric collections of workers. For example, when one process is dramatically faster than another, it helps to have multiple copies of the slow processes to keep up with the faster process. This is handled politely by the `concurrent.futures` package, which lets an application create a "pool" of workers.

The pool can be threads or processes, depending on the nature of the work. For the most part, CPU cores tend to be used better by process pools, because OS scheduling is often process-focused. The Python **Global Interpreter Lock (GIL)** often prohibits compute-intensive thread pools from making effective use of CPU resources.

For huge data sets, worker-pool architecture can provide some performance improvements. There is overhead in serializing and deserializing

the Python objects to pass the values from process to process. This overhead imposes some limitations on the benefits of multiprocessing.

The components that implement a worker process pool are shown in [Figure 10.3](#).

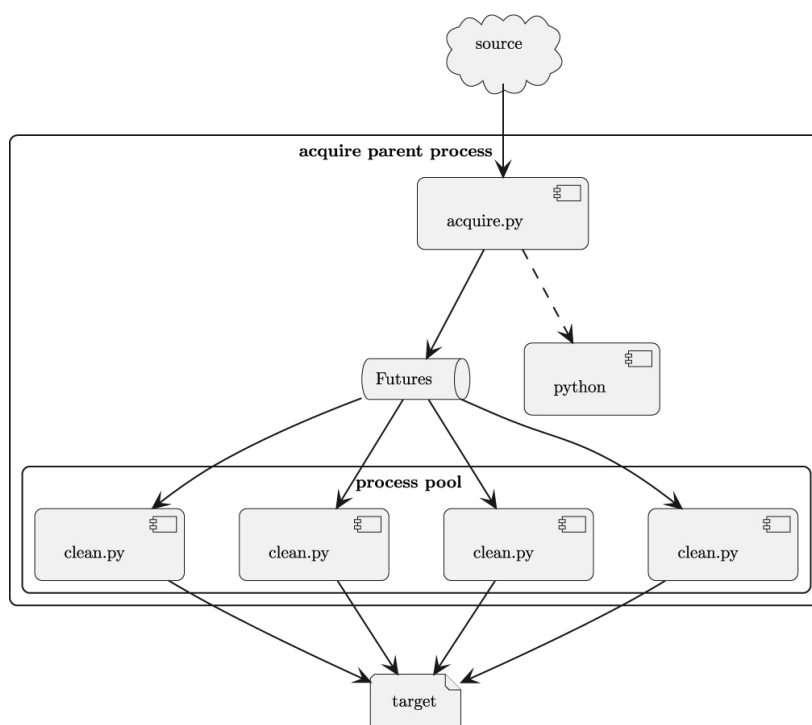


Figure 10.3: Components of a worker pool

This design is a significant alteration to the relationship between the `acquire.py` and `clean.py` applications. When the `acquire.py` application creates the process pool, it uses class and function definitions available within the same parent process.

This suggests the `clean.py` module needs to have a function that processes exactly one source document. This function may be as simple as the following:

```

from multiprocessing import get_logger

import acquire_model
import analysis_model

def clean_sample(
    acquired: acquire_model.SeriesSample
) -> analysis_model.SeriesSample:
    try:
        return analysis_model.SeriesSample.from_acquire_dataclass(acquired)
    except ValueError as ex:
        logger = get_logger()
        logger.error("Bad sample: %r\n%r\n", acquired, ex)
        return None

```

This function uses the analysis model definition, `SeriesSample`, to perform the validation, cleaning, and conversion of the acquired data. This can raise exceptions, which need to be logged.

The child processes are created with copies of the parent application's logging configuration. The `multiprocessing.get_logger()` function will retrieve the logger that was initialized into the process when the pool of worker processes was created.

The `acquire.py` application can use a higher-order `map()` function to allocate requests to the workers in an executor pool. The general approach is shown in the following incomplete code snippet:

```
with target_path.open('w') as target_file:
    with concurrent.futures.ProcessPoolExecutor() as executor:
        with source_path.open() as source:
            acquire_document_iter = get_series(
                source, builder
            )
            clean_samples = executor.map(
                clean.clean_sample,
                acquire_document_iter
            )
            count = clean.persist_samples(target_file, clean_samples)
```

This works by allocating a number of resources, starting with the target file to be written, then the pool of processes to write clean data records to the file, and finally, the source for the original, raw data samples. Each of these has a context manager to be sure the resources are properly released when all of the processing has finished.

We use the `ProcessPoolExecutor` object as a context manager to make sure the subprocesses are properly cleaned up when the source data has been fully consumed by the `map()` function, and all of the results retrieved from the `Future` objects that were created.

The `get_series()` function is an iterator that provides the builds the acquire version of each `SeriesSample` object. This will use an appropriately configured `Extractor` object to read a source and extract a series from it.

Since generators are lazy, nothing really happens until the values of the `acquire_document_iter` variable are consumed. The `executor.map()` will consume the source, providing each document to the pool of workers to create a `Future` object that reflects the work being done by a separate subprocess. When the work by the subprocess finishes, the `Future` object will have the result and be ready for another request.

When the `persist_samples()` functions consume the values from the `clean_samples` iterator, each of the `Future` objects will yield their result. These result objects are the values computed by the

`clean.clean_sample()` function. The sequence of results is written to the target file.

The `concurrent.futures` process pool `map()` algorithm will preserve the original order. The process pool offers alternative methods that can make results ready as soon as they're computed. This can reorder the results; something that may or may not be relevant for subsequent processing.